# r_tutorial_week_02

**Werkcollege 2**

Sebastian A. J. Lortz

15 October 2026

## Table of contents

**How to use this tutorial** **2**

**1 Start workflow** **2**
Your turn . . . 3

**2 Getting the data ready** **3**
Your turn . . . 3

**3 Loading the data** **3**
Your turn . . . 4

**4 Data frames** **4**
Your turn . . . 5

**5 Looking at your data** **5**
Your turn . . . 7

**6 Cleaning and changing your data** **7**
6.1 Variable classes . . . 7
Your turn . . . 8
6.2 Filtering: choosing rows . . . 8
Your turn . . . 8
6.3 Removing columns . . . 9
Your turn . . . 9
6.4 Making a new variable . . . 9
Your turn . . . 9
6.5 Putting values into categories . . . 10
Your turn . . . 10

**7 Describing your sample** **11**
7.1 Counting categories with table() . . . . . 11
7.2 Numbers in short: mean and sd . . . . . 11
7.3 Histograms with hist() . . . . . 11
Your turn . . . . . 12
7.4 Boxplots with boxplot() . . . . . 13
Your turn . . . . . 14

**8 Saving your cleaned data** **14**
Your turn . . . . . 15

**9 Recap: what you can now do** **15**

## How to use this tutorial

Welcome back! Last week you learned some basics including objects, vectors, indexing, and functions. This week we work with a **dataset**: a whole table of data, like the answers from our class questionnaire.

Work through this page carefully, from top to bottom. Every short explanation is followed by a **Your turn** task to do yourself.

General tips:

- Read each part slowly and in order. Each step builds on the one before it.
- Type the code yourself instead of copying and pasting it. You learn faster this way.
- Errors are normal. Read the message and try again.
- If you are stuck, check the FAQ or ask a teacher.
- Save your work often.

---

## 1 Start workflow

Based on last week's tutorial, we **always** start with the following **start workflow**:

1. Open **RStudio**: click on RStudio application.
2. Set the **working directory**: click *Session* > *Set Working Directory* > *Choose Directory*.
3. Open a **new script**: click *File* > *New File* > *R Script*. Or, open an **existing script**: click *File* > *Open File* > and navigate to the file.
4. On the first line, write a comment with the title, for example: `# R tutorial - Week 2`.
5. Save the script as `week2.R` in your `research_methods` folder.

Today we work in a **new script**. However, often we need to pick-up previous work and open an existing script.

#### Your turn

Complete the **start workflow** for working with a new script.

---

## 2 Getting the data ready

Our class answered a short questionnaire. The answers were collected, made **anonymous** (no names, only a number per person), and saved as a **CSV file**. A CSV file is a simple table where the values are separated by commas. Almost every program can open it.

#### Your turn

1. Download the file `class_survey.csv` from **Brightspace**.
2. Move it into the `research_methods` folder (the same folder from the structure we made in Week 1).

---

## 3 Loading the data

**RStudio** does not know about the file until you load it. We load a CSV with the function `read.csv()`. Inside the brackets we put the **path** to the file, in quotes. Because our working directory is `research_methods`, and the file sits directly in that folder, the path is just `"class_survey.csv"`.

We store the table in an object so we can use it again. We will call it `survey`.

```
survey <- read.csv("class_survey.csv")
```

The table is now stored. Look at the **Environment** pane (top-right): `survey` appears, with the number of rows (people) and columns (questions).

⚠ Watch out

If you see `Error: cannot open file ... No such file or directory`, R cannot find the file. Check two things:

1. your working directory is `research_methods`: run `getwd()` to print the current working directory
2. the file really sits in your `research_methods` folder and is spelled exactly, including `.csv`: in the bottom-right pane you can click on files and navigate to the folder.

💡 Going further (optional): Excel files

R can open almost any kind of data file, but some formats need an extra **package**. `read.csv()` works for CSV files. Excel files (`.xlsx` or `.xls`), for example, need a different package, such as `readxl` with its function `read_excel()`. For now, the easiest path is to open the Excel file once and save it as a CSV, so you can use `read.csv()`.

#### Your turn

Load the data into an object called `survey`, then check that `survey` appears in the Environment pane.

---

## 4 Data frames

The object `survey` is a **data frame**. A data frame is R's word for a table. In our questionnaire data:

- each **row** is one **participant** (one person who answered). Row 1 is the first person, row 2 the second, and so on.
- each **column** is one **variable** (one question or measure). For example, `age` is one column, `self_efficacy` is another.

So one row holds *all the answers of one person*, and one column holds *one answer from everybody*. You can think of a data frame as several vectors (from Week 1) standing side by side, one per column.

To use a single column, write the data frame name, then a **dollar sign** `$`, then the column name:

```
survey$age   # all the ages, as a vector
#>  [1] 25 17 22 20 21 18 24 19 17 22 18 18 21 24 21 19 19 19 20 24 18 18 25 21 25
#> [26] 18 25 23 18 21
```

A column is just a vector, so everything you learned last week still works:

```
mean(survey$age)        # average age
#> [1] 20.66667
survey$age[1]           # the first person's age
#> [1] 25
survey$age[survey$age > 20]   # ages above 20
#>  [1] 25 22 21 24 22 21 24 21 24 25 21 25 25 23 21
```

You can also pick a value by its row and column, inside square brackets, with a **comma** in between: `survey[row, column]`.

```
survey[1, ]    # the whole first row (a comma, then nothing = all columns)
#>   id age self_efficacy study_hours program
#> 1  1  25            40           8       1
survey[1, 2]   # row 1, column 2
#> [1] 25
```

#### Your turn

Print the `self_efficacy` column. Then print the `self_efficacy` score of the third person, using `$` and square brackets. Also, print the minimum and maximum `self_efficacy` score.

---

## 5 Looking at your data

Before changing anything, always look at your data first. Here are the most useful commands.

How big is the table?

```
nrow(survey)   # number of rows (people)
#> [1] 30
ncol(survey)   # number of columns (variables)
#> [1] 5
```

What are the columns called?

```
names(survey)
#> [1] "id"            "age"           "self_efficacy" "study_hours"
#> [5] "program"
```

See the first few rows:

```
head(survey)
#>   id age self_efficacy study_hours program
#> 1  1  25            40           8       1
#> 2  2  17            31          18       2
#> 3  3  22            24           6       1
#> 4  4  20            31          14       3
#> 5  5  21            35          13       1
#> 6  6  18            30          16       2
```

Get a quick summary of every variable with `summary()`:

```
summary(survey)
#>        id              age        self_efficacy    study_hours
#>  Min.   : 1.00   Min.   :17.00   Min.   :22.00   Min.   : 2.00
#>  1st Qu.: 8.25   1st Qu.:18.00   1st Qu.:29.25   1st Qu.: 8.00
#>  Median :15.50   Median :20.50   Median :33.00   Median :13.50
#>  Mean   :15.50   Mean   :20.67   Mean   :33.23   Mean   :12.17
#>  3rd Qu.:22.75   3rd Qu.:22.75   3rd Qu.:38.00   3rd Qu.:16.00
#>  Max.   :30.00   Max.   :25.00   Max.   :44.00   Max.   :22.00
#>     program
#>  Min.   :1.000
#>  1st Qu.:1.000
#>  Median :2.000
#>  Mean   :2.033
#>  3rd Qu.:3.000
#>  Max.   :3.000
```

For numbers, `summary()` shows the smallest value, the 1st Quartile, the median, the average, the 3rd Quartile, and the largest value. Because R still thinks `program` is a number, it even shows its "average", which makes no sense for a category. Another reason to recode it later.

Finally, you can open the table in a spreadsheet view. Type this in RStudio (the capital **V** matters):

```
View(survey)
```

> ⚠ Watch out
>
> It is `View()` with a capital **V**. Remember from Week 1: R is case-sensitive, so `view()` with a small v will not work.

#### Your turn

Use these commands on `survey`: `nrow()`, `ncol()`, `names()`, and `summary()`. How many people answered the questionnaire? How many variables are in the dataset? What is the average age?

---

## 6 Cleaning and changing your data

Real data almost always needs some tidying before we describe it. We will keep the loaded data in `survey` and change it step by step.

### 6.1 Variable classes

Every column has a **class**: the kind of values it holds. The most common classes are:

- **numeric**: numbers, for example `age`.
- **character**: text (always in quotes), for example a label like `"yes"`.
- **logical**: `TRUE` or `FALSE` values.
- **factor**: categories, for example a study programme.

You check the class of a column with the function `class()`:

```
class(survey$age)       # age holds whole numbers
#> [1] "integer"
class(survey$program)   # program is stored as whole numbers too
#> [1] "integer"
```

> **integer or numeric?**
>
> For whole numbers R often says `integer`; for numbers with decimals it says `numeric`. Both are simply **numbers**, so you do not need to worry about the difference for now.

Look closely at `program`. R stores it as a number, because the questionnaire used the codes 1, 2, and 3. But `program` is not really a number: each code stands for a **category** (one of three study programmes). This is why its "average" in `summary()` made no sense. Knowing the class of a variable tells you what you can sensibly do with it: you can take the mean of a real number, but not of a category.

#### Your turn

Check the class of the `study_hours` column and of the `self_efficacy` column with `class()`. Which column in the data set is stored as a number, but is really a **category**?

### 6.2 Filtering: choosing rows

**Filtering** means keeping only the rows that match a rule. We use square brackets with a condition before the comma: `survey[condition, ]`. The empty part after the comma means "keep all columns".

```
adults <- survey[survey$age >= 18, ]   # keep only people aged 18 or older
nrow(adults)                           # how many are left?
#> [1] 28
```

Read it as: "keep the rows of `survey` where `age` is 18 or higher."

> **Watch out**
>
> Do not forget the **comma**. `survey[survey$age >= 18, ]` keeps rows. Without the comma, R gets confused, because it cannot tell rows from columns.

#### Your turn

Create an object called `keen` that keeps only the people who study more than 15 hours per week (`study_hours > 15`). How many people are in `keen`?

## 6.3 Removing columns

Sometimes a column is not useful for analysis. The `id` number, for example, only tells people apart; it is not something we study. We remove a column by setting it to `NULL`:

```
survey$id <- NULL   # remove the id column
names(survey)       # id is gone
#> [1] "age"           "self_efficacy" "study_hours"   "program"
```

### Your turn

Look at the column names again with `names(survey)` and check that `id` is no longer there. What other function could you use to check this?

## 6.4 Making a new variable

We can build a new column from the columns we already have. Give the new column a name with `$`, and put a calculation on the right of the arrow `<-`.

```
survey$study_per_day <- survey$study_hours / 7   # weekly hours -> daily hours
head(survey)
#>   age self_efficacy study_hours program study_per_day
#> 1  25            40           8       1     1.1428571
#> 2  17            31          18       2     2.5714286
#> 3  22            24           6       1     0.8571429
#> 4  20            31          14       3     2.0000000
#> 5  21            35          13       1     1.8571429
#> 6  18            30          16       2     2.2857143
```

The new column `study_per_day` now appears in the table.

### Your turn

Make a new variable `study_per_day_rounded` that rounds `study_per_day` to whole numbers, using the function `round()`. Then look at the first rows.

## 6.5 Putting values into categories

**Categorizing** means turning a number into groups. A common tool is `ifelse()`. It takes three parts: a test, the value if the test is `TRUE`, and the value if it is `FALSE`.

```
survey$efficacy_level <- ifelse(survey$self_efficacy >= 32, "higher", "lower")
table(survey$efficacy_level)
#>
#> higher  lower
#>     16     14
```

The function `table()` **counts** how often each value appears. Here it shows how many people are in the `"higher"` group and how many in the `"lower"` group. We use `table()` a lot when describing categories, and look at it again in the next section.

### Your turn

Make a variable `studies_a_lot` that is `"yes"` when `study_hours` is 14 or more, and `"no"` otherwise. Then count the groups with `table()`.

**Recoding** means changing values into clearer ones. Our `program` column holds the codes 1, 2, and 3, but those numbers are hard to read. We can make a new column with real names. We start it as `NA` (R's word for "not available / empty") and then fill it in:

```
survey$program_name <- NA   # a new, empty column
survey$program_name[survey$program == 1] <- "Pedagogy"
survey$program_name[survey$program == 2] <- "Psychology"
survey$program_name[survey$program == 3] <- "Sociology"
table(survey$program_name)
#>
#>   Pedagogy Psychology  Sociology
#>         10          9         11
```

Read one of those lines as: "for the rows where `program` equals 1, set `program_name` to Pedagogy." We kept the original `program` column, which is good practice: never throw away the raw values. As before, testing "is equal to" needs **two** equals signs `==`.

---

# 7 Describing your sample

Now that the data is clean, we describe it. This is the first step of almost every study: who is in the sample, and how are the values spread out?

## 7.1 Counting categories with table()

For categories (text variables), we count how often each value appears with `table()`.

```
table(survey$program_name)
#>
#>   Pedagogy Psychology  Sociology
#>         10          9         11
```

## 7.2 Numbers in short: mean and sd

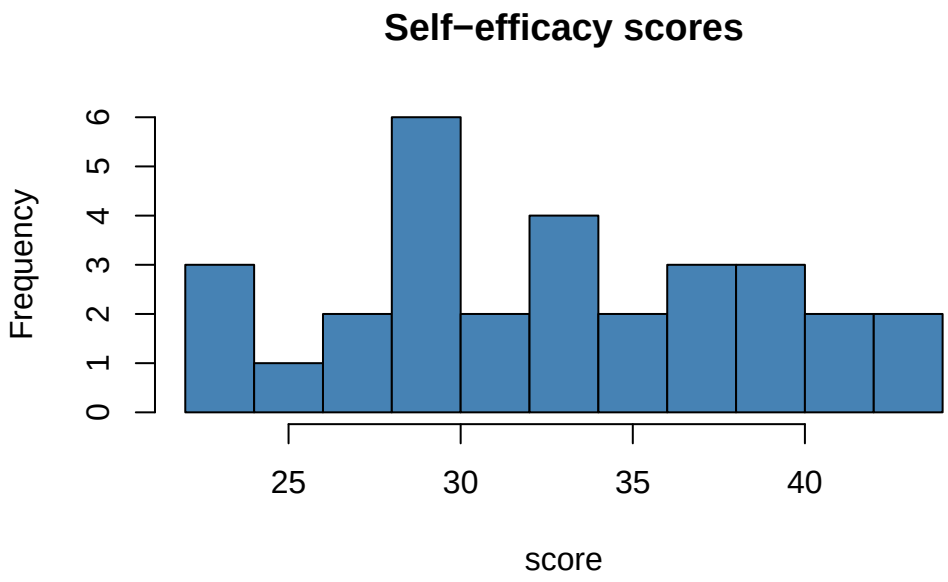
For numbers, the average (`mean()`), the median (`median()`), and the standard deviation (`sd()`) give a quick picture.

```
mean(survey$self_efficacy)   # average self-efficacy
#> [1] 33.23333
sd(survey$self_efficacy)     # how spread out the scores are
#> [1] 6.043996
```

## 7.3 Histograms with hist()

A **histogram** shows how a numeric variable is distributed. It puts the values into bars: tall bars mean many people have those values.

```
hist(survey$self_efficacy,
     breaks = 10,                    # roughly how many bars
     col = "steelblue",              # colour of the bars
     main = "Self-efficacy scores",  # title
     xlab = "score")                 # label for the x-axis
```




Each part inside the brackets is an **argument** that changes the plot: `main` renames the title, `xlab` renames the x-axis label, `breaks` sets roughly how many bars there are, and `col` sets the colour of the bars. Try changing them and run the line again to see the effect.

> Going further (optional): nicer graphics with ggplot2
>
> The plots from `hist()` are fine for a quick look. For nicer graphics, many people use the **package** `ggplot2`. You install it once and load it each session:
>
> ```
> install.packages("ggplot2")   # once
> library(ggplot2)              # each session
>
> ggplot(survey, aes(x = self_efficacy)) +
>   geom_histogram(fill = "steelblue", colour = "white", bins = 10) +
>   labs(title = "Self-efficacy scores", x = "score", y = "count")
> ```
>
> You do not need `ggplot2` for this course, but it is good to know it exists.

### Your turn

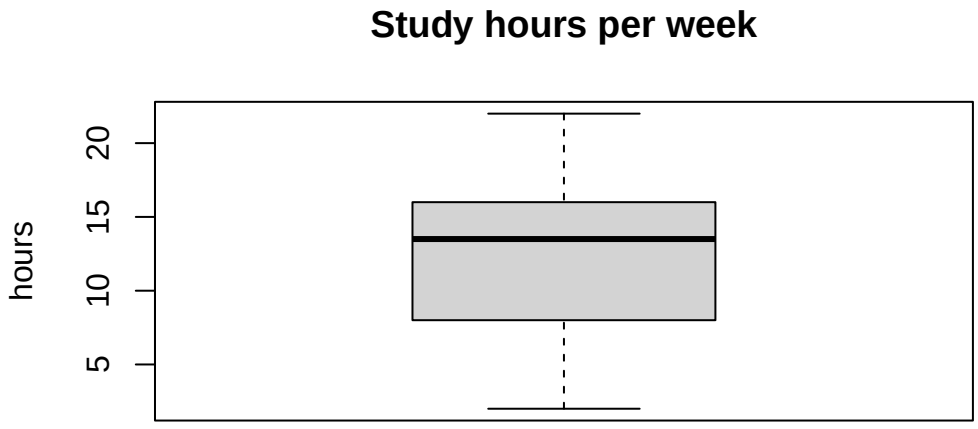
Make a histogram of `age`. Give it a clear title and an x-axis label, and try a colour of your choice with `col` and a different number of `breaks`. Then print the mean and median of `age`:

are they similar or different? Argue why, and relate your answer to the histogram you just plotted.

### 7.4 Boxplots with boxplot()

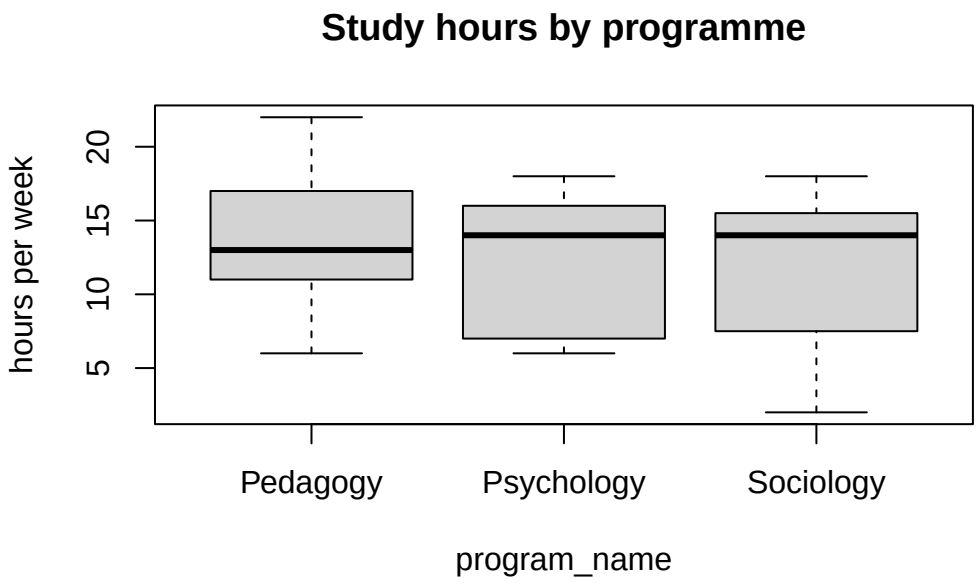
A **boxplot** is another way to show the distribution of a numeric variable. The thick line is the middle value (the median), the box holds the middle half of the people (the interquartile range), and the lines reach out to the smallest and largest typical values.

```
boxplot(survey$study_hours,
        main = "Study hours per week",
        ylab = "hours")
```




Boxplots are useful when you **compare groups**. We do that with a tilde ~, which you read as "by". The variable before the ~ is the number; the variable after it is the group.

```
boxplot(study_hours ~ program_name,
        data = survey,
        main = "Study hours by programme",
        ylab = "hours per week")
```




Now you can see whether students in some programmes study more than in some others in your sample.

#### Your turn

Make a boxplot of `self_efficacy` **by** `program_name`. Which programme has the highest middle value? What can you conclude from it?

---

## 8 Saving your cleaned data

You changed the data, so save the clean version under a **new** name. This keeps the original file safe. We use `write.csv()` and add `row.names = FALSE` so R does not add an extra column of row numbers.

```
write.csv(survey, "data/class_survey_clean.csv", row.names = FALSE)
```

**Your turn**

Save your cleaned `survey` as `class_survey_clean.csv` in your `research_methods` folder. Then check in the **Files** pane (bottom-right) that the new file is there. Also save your script.

---

## 9 Recap: what you can now do

Last week you handled single numbers and vectors. You can now work with a whole dataset. You can:

- load a CSV file with `read.csv()` and store it as a data frame;
- reach a single column with `$`, and a single cell with `[row, column]`;
- inspect data with `nrow()`, `ncol()`, `names()`, `head()`, `summary()`, `View()`, and check a column's type with `class()`;
- filter rows with a condition;
- make new variables and remove ones you do not need;
- recode values into clear labels and put numbers into categories;
- describe your sample with `table()`, `mean()`, `sd()`, `hist()`, and `boxplot()`;
- save a cleaned dataset with `write.csv()`.

A small cheat sheet to keep:

| You want to… | You write… |
|---|---|
| Load a CSV | `read.csv("data/file.csv")` |
| View a data frame | `View(df)` |
| Use one column | `df$age` |
| Rows and columns count | `nrow(df)`, `ncol(df)` |
| First rows | `head(df)` |
| Class of a column | `class(df$x)` |
| Summary of all variables | `summary(df)` |
| Keep some rows (filter) | `df[df$age >= 18, ]` |
| Make a new variable | `df$new <- df$x / 7` |
| Remove a variable | `df$old <- NULL` |
| Recode a value | `df$x[df$x == 1] <- "label"` |
| Category from a rule | `ifelse(df$x >= 32, "higher", "lower")` |
| Count a category | `table(df$group)` |
| Histogram | `hist(df$x)` |
| Boxplot by group | `boxplot(y ~ group, data = df)` |
| Save a CSV | `write.csv(df, "data/out.csv", row.names = FALSE)` |

Well done. Keep your materials in the `research_methods` folder, we build directly on them next week.

---